

СИСТЕМНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

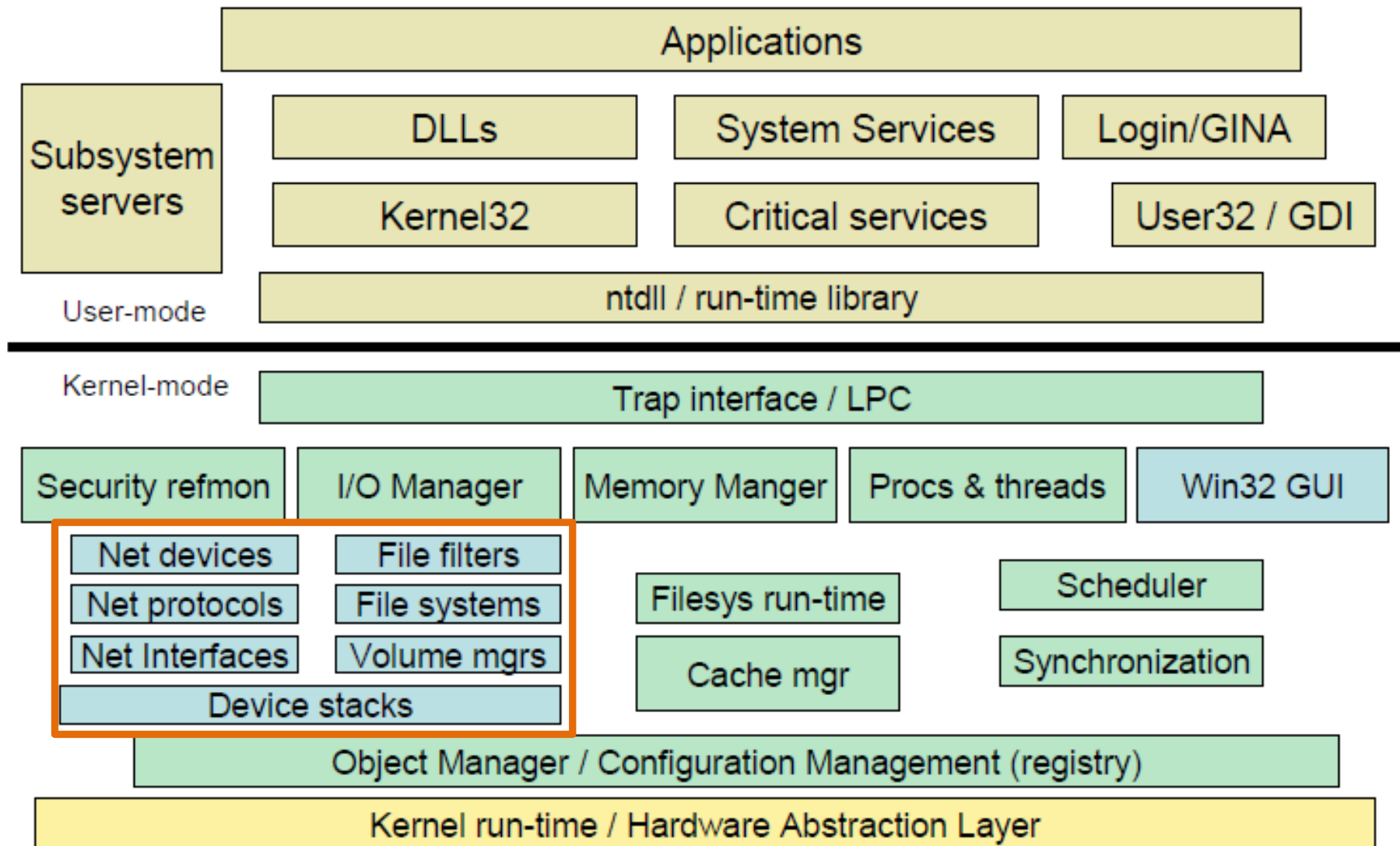
БЛОК 2

Ляхманов Д.А.
к.т.н., доцент каф. ИСУ

2018г.

ПРОГРАММИРОВАНИЕ УРОВНЯ ЯДРА WINDOWS

СТРУКТУРА ЯДРА WINDOWS



ТИПЫ ДРАЙВЕРОВ

порт-драйверы (port-driver)

- определяет абстрактный специфичный интерфейс устройства
- обеспечивает доступ к функциям минипорт-драйвера

минипорт-драйверы (miniport-driver)

- реализуют специфичный интерфейс физического устройства
- инкапсулируют логику работы устройства

класс-драйверы (class-driver)

- определяют абстрактный системный интерфейс

драйверы шин (bus-driver)

фильтр-драйверы (filter-driver)

- осуществляют обработку данных от устройств или драйверов приложений

драйверы приложений (software-driver)

- не ассоциированы с физическими устройствами
- используются для доступа к закрытым данным ядра

ТЕХНОЛОГИИ РЕАЛИЗАЦИИ

WDM (Windows Driver Model) – фреймворк, определяющий унифицированную модель драйвера, введен для Windows 98 и старше

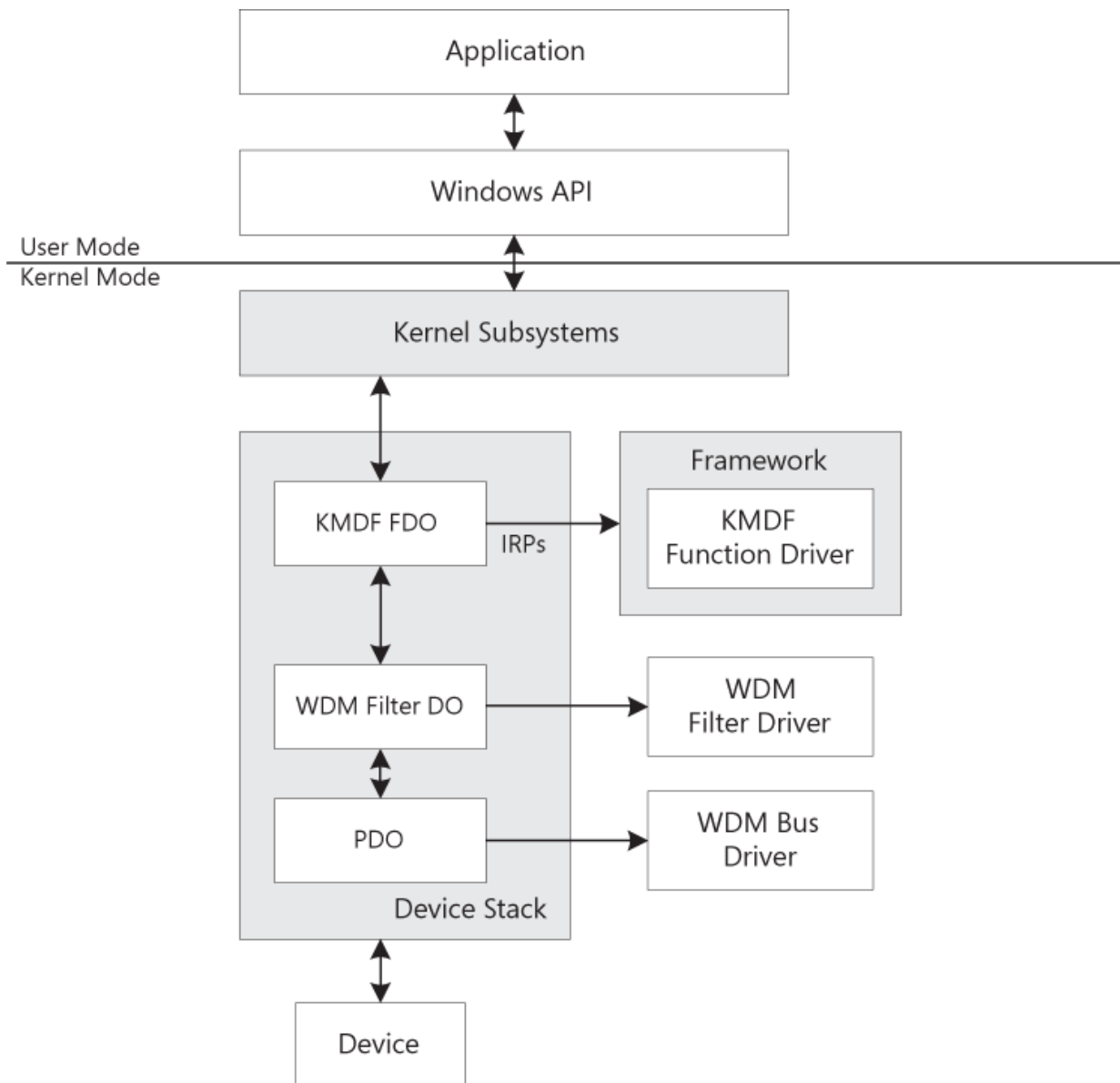
- слишком сложен для освоения
- требует знания технических особенностей устройства
- требуется большое количество сопровождающего кода
- не поддерживает создание UserMode-драйверов

WDF (Windows Driver Framework) – фреймворк, реализующий упрощенную модель программирования драйвера, является надстройкой на WDM

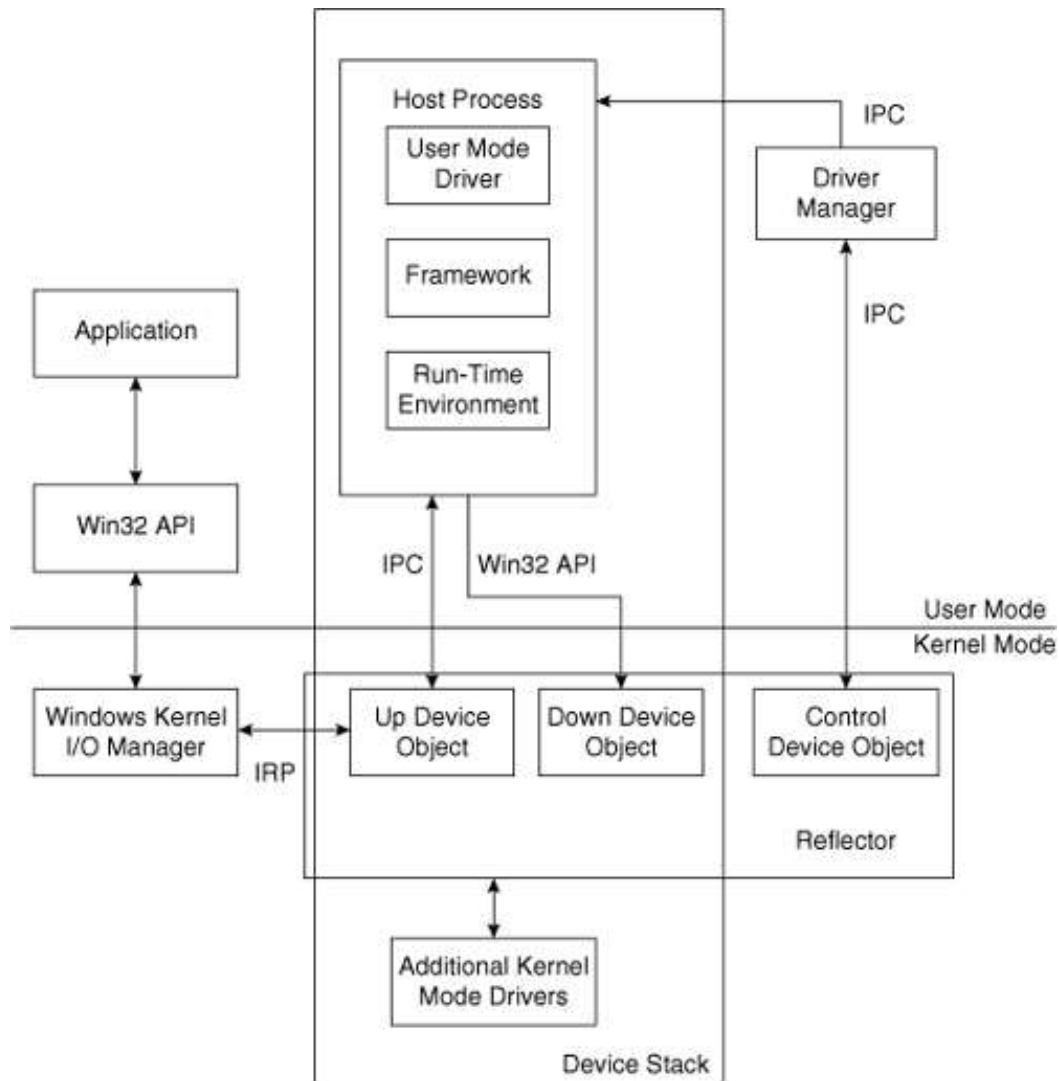
KMDF (Kernel-Mode Driver Framework) – фреймворк для разработки драйверов режима ядра

UMDF (User-Mode Driver Framework) – фреймворк для разработки драйверов пользовательского режима

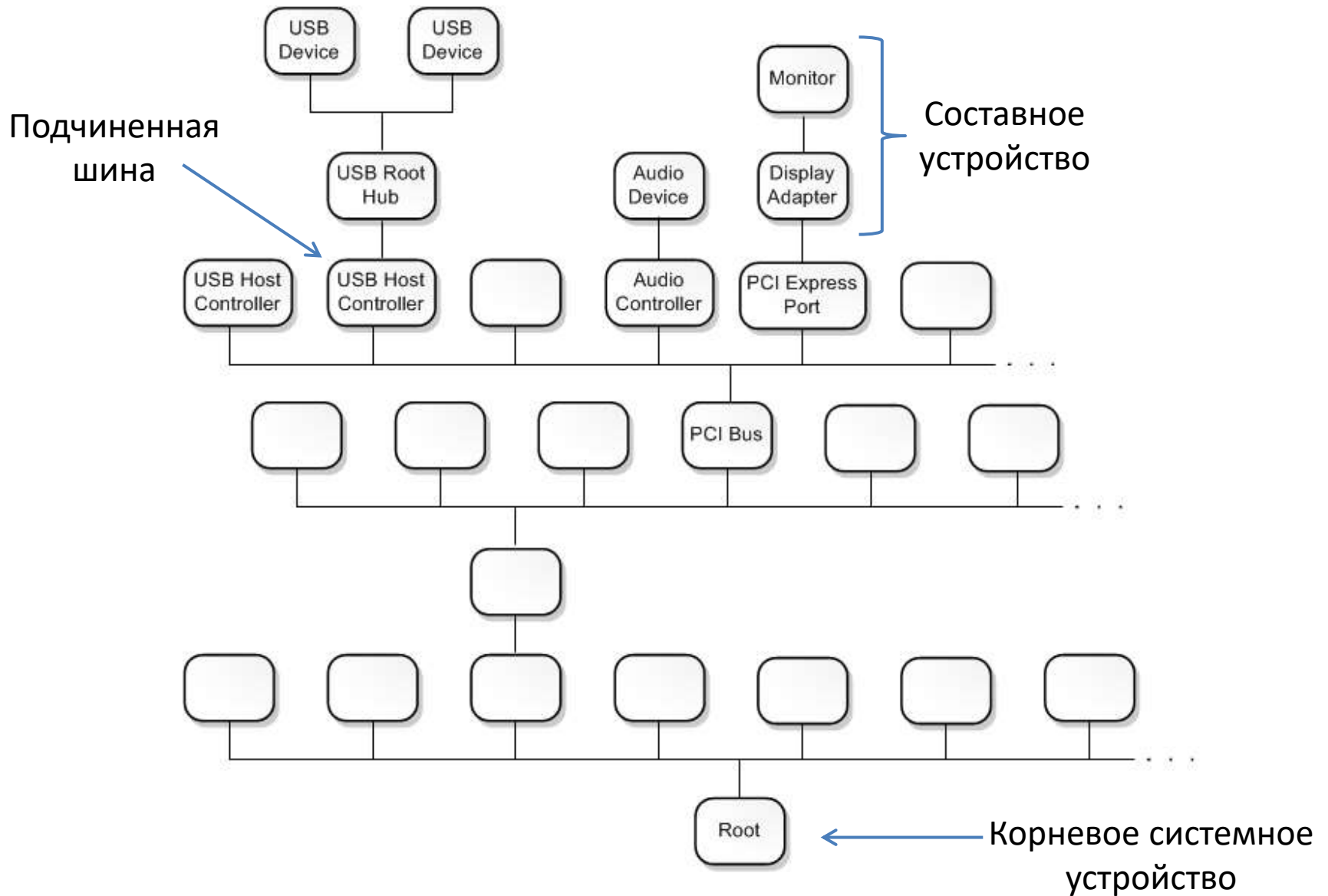
РАЗМЕЩЕНИЕ KMDF-ДРАЙВЕРА



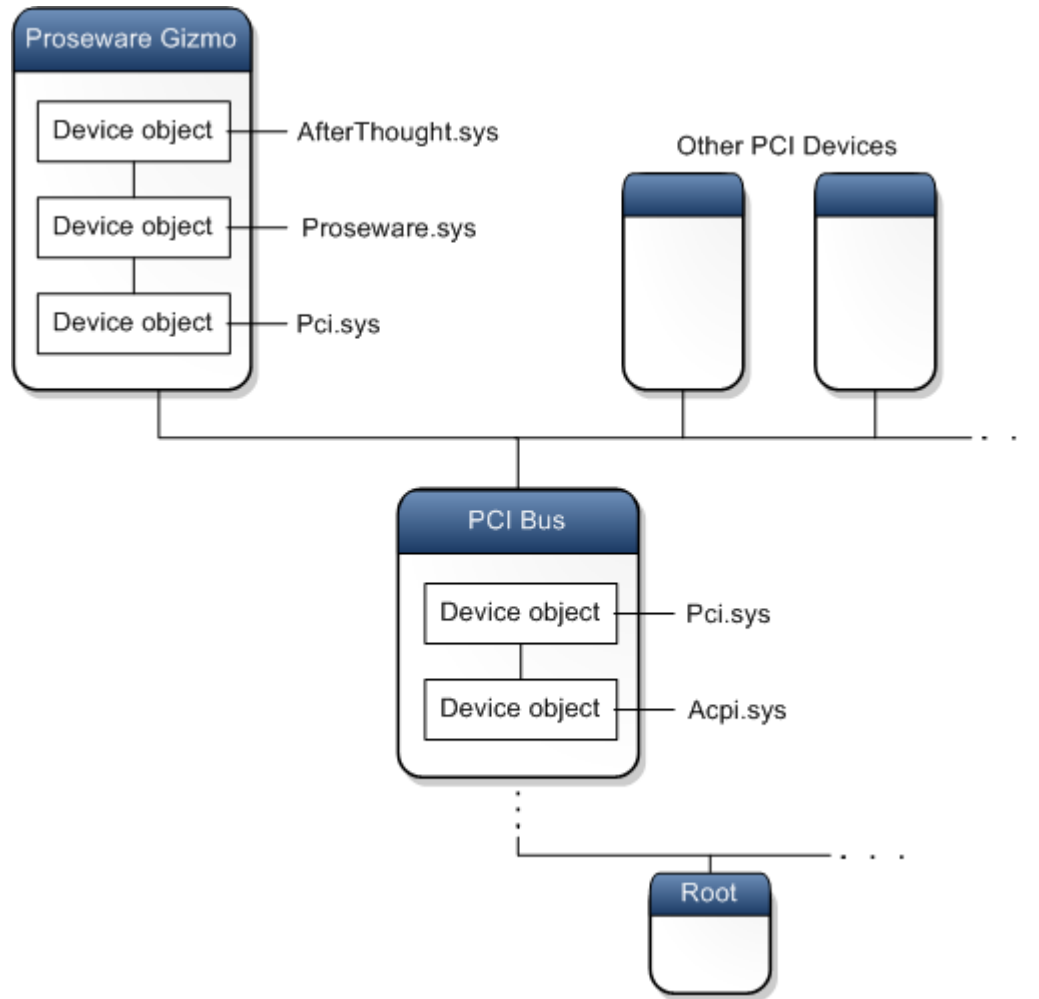
РАЗМЕЩЕНИЕ UMDF-ДРАЙВЕРА



СТРУКТУРИРОВАНИЕ PnP-УСТРОЙСТВ

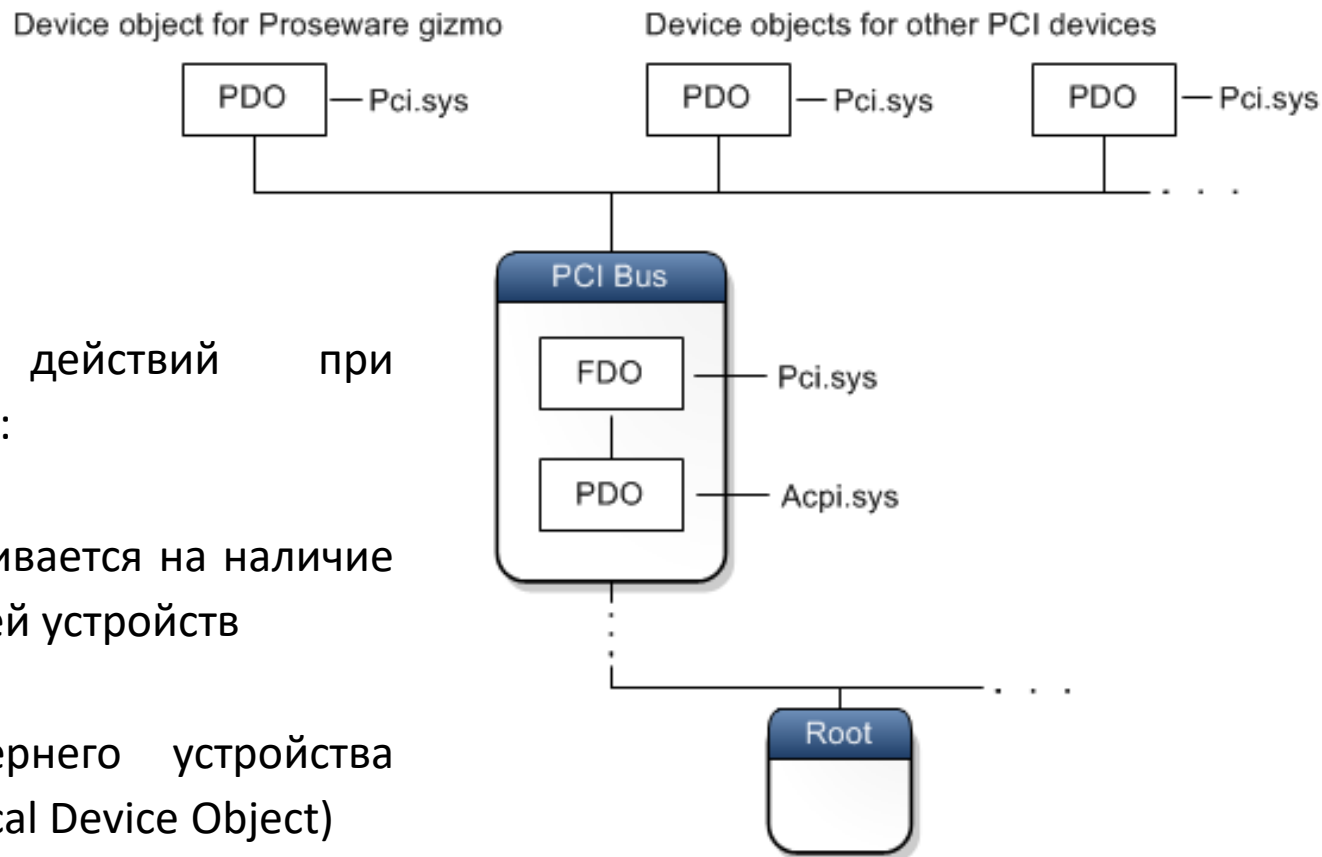


СТЕК УСТРОЙСТВ



1. Каждая нода устройства из PnP-дерева имеет структурированный список из объектов устройств, называемый **стеком устройств** (device stack)
2. Каждому объекту устройства соответствует драйвер
3. сочетание драйвера и объекта устройства называется **парой устройства** (device pair)
4. Устройства, присоединенные к шине могут иметь в составе своего стека функциональный драйвер шины

ФОРМИРОВАНИЕ СТЕКА УСТРОЙСТВ (1)

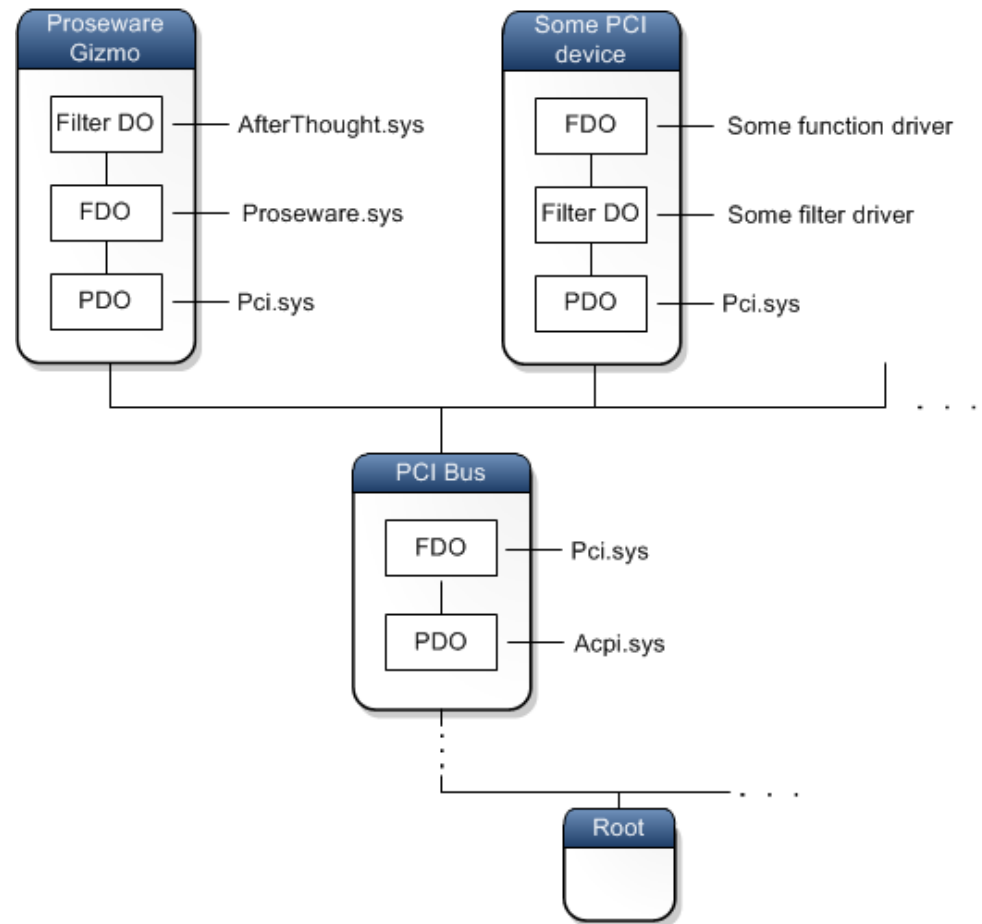


Последовательность действий при построении PnP-дерева:

1. Каждая шина опрашивается на наличие присоединенных к ней устройств
2. Для каждого дочернего устройства создается **PDO** (Physical Device Object)
3. Каждому PDO соответствует интерфейсный драйвер шины (**Pci.sys**)

ФОРМИРОВАНИЕ СТЕКА УСТРОЙСТВ (2)

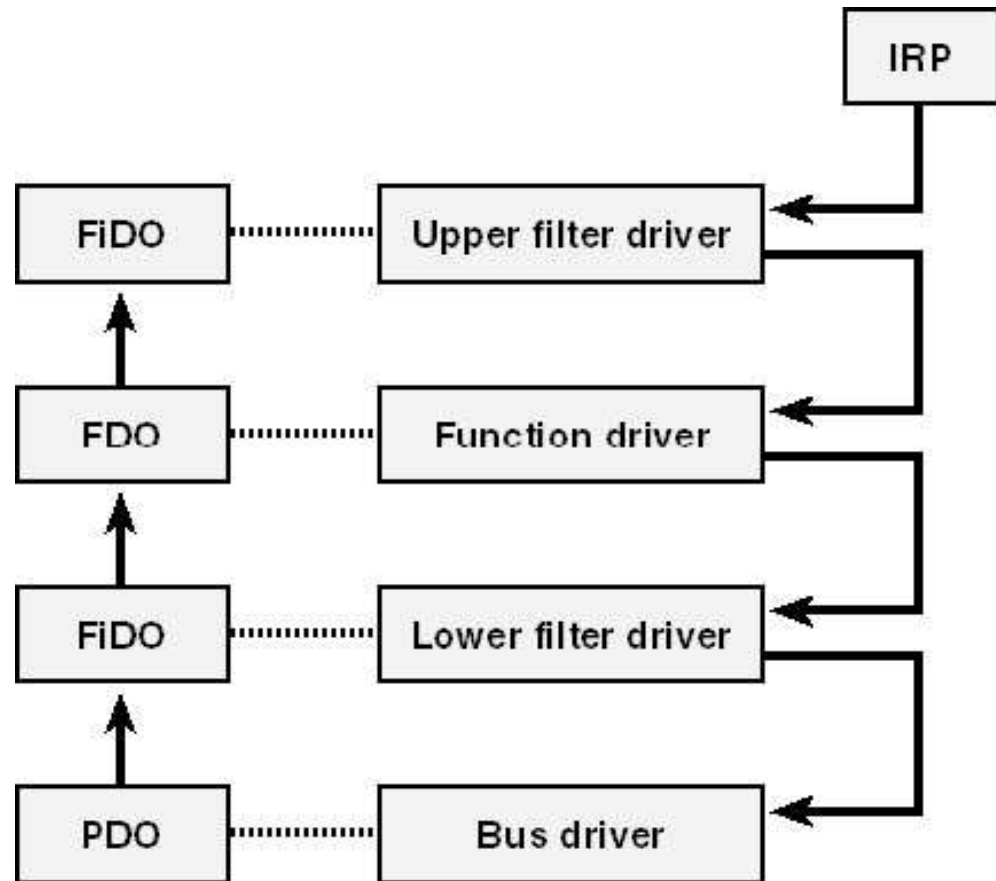
4. Для каждого PDO создается PnP-нода и стек устройств
5. Используя реестр ОС находит функциональный драйвер и создает для него объект устройства (**FDO**);
6. Находятся все фильтр-драйверы, для каждого из них создаются объекты (**Filter DO**) и добавляются к стеку



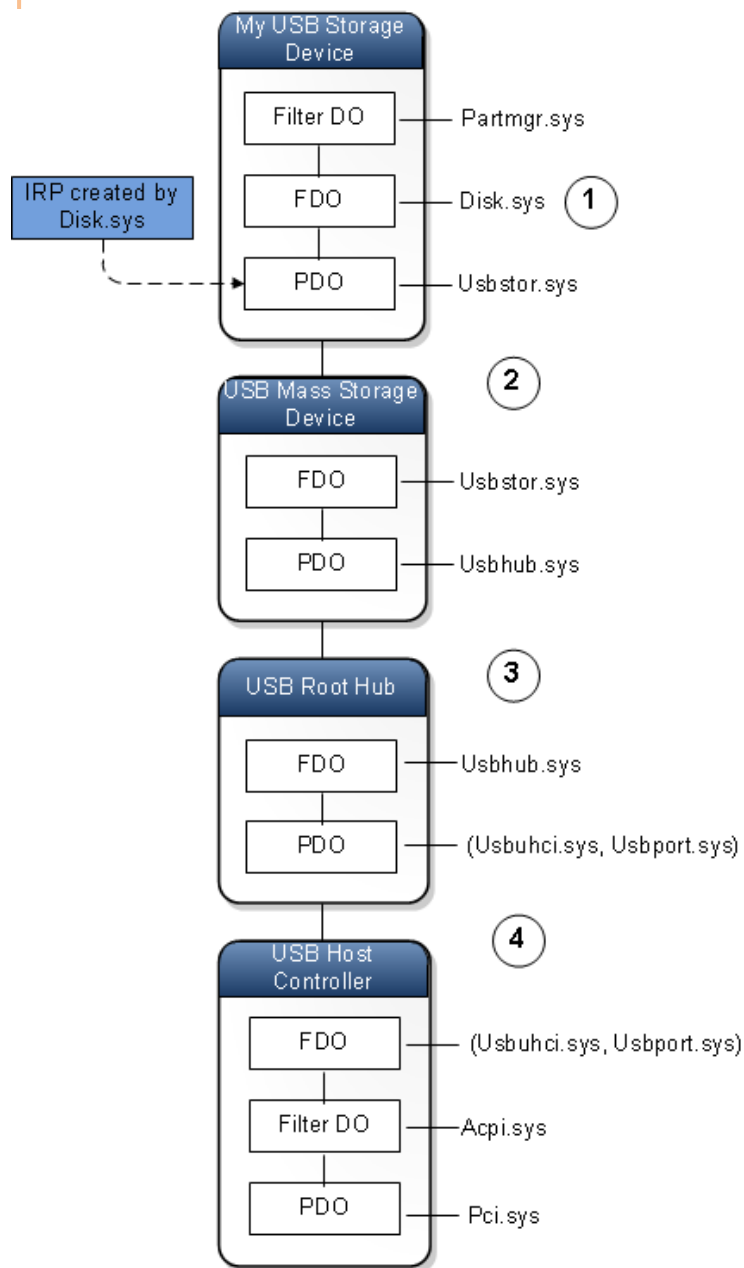
Только функциональный драйвер осуществляет прямое управление устройством; FDO всегда один в стеке

ВЗАИМОДЕЙСТВИЕ МЕЖДУ ДРАЙВЕРАМИ

IRP (Interrupt Request Package) – информационная структура для передачи и последовательной обработки данных между физическими устройствами и программной средой компьютера.



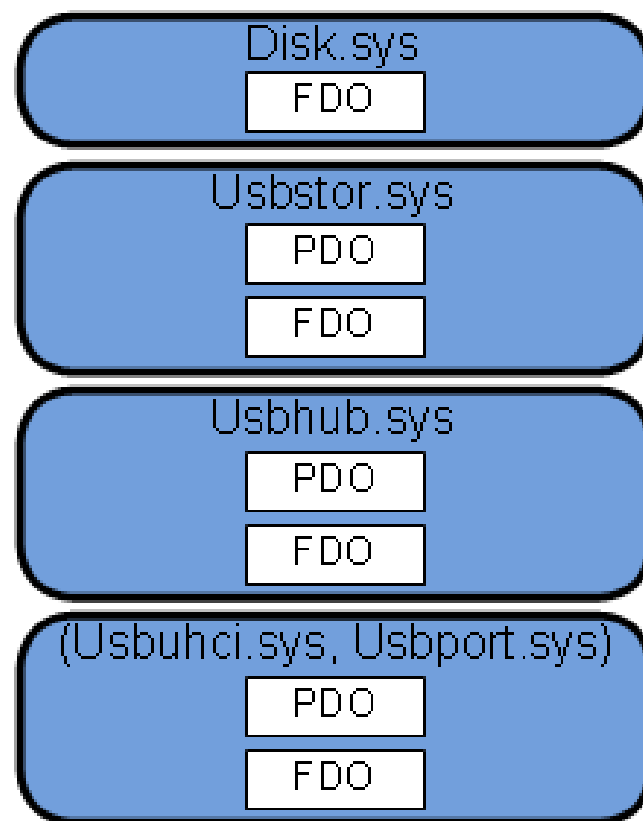
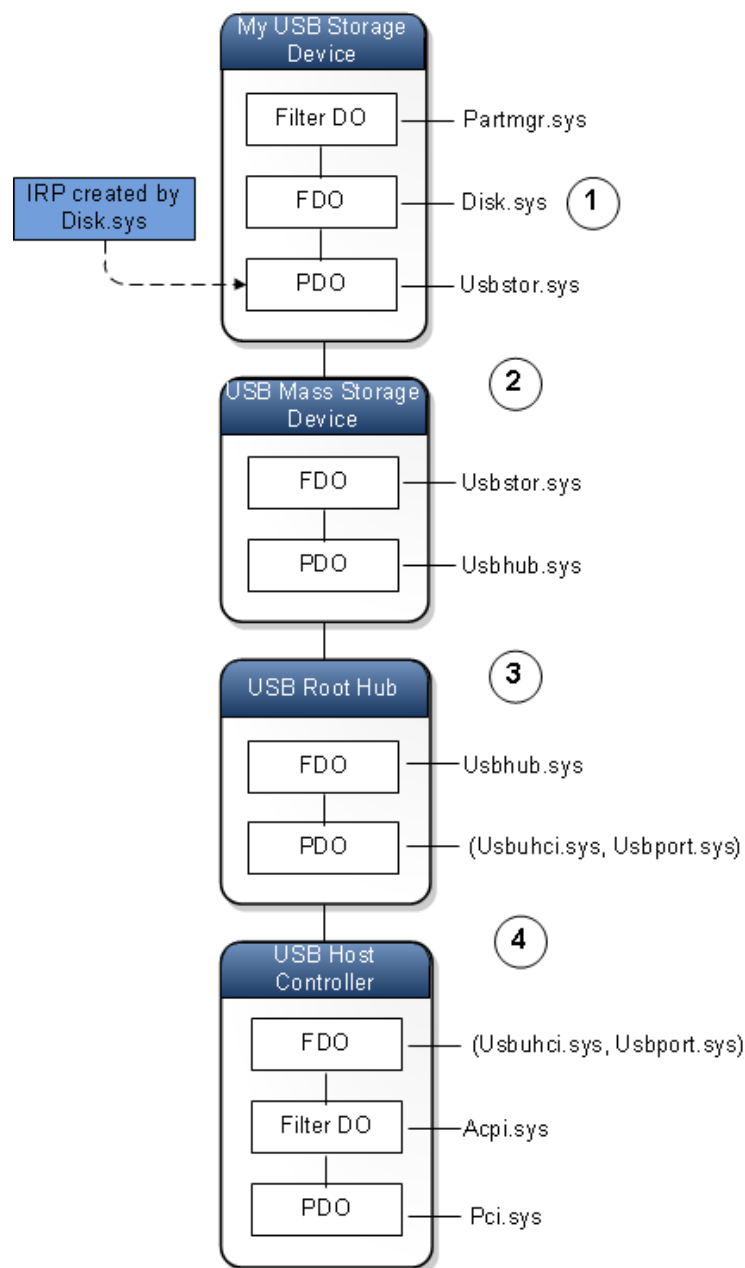
ОБРАБОТКА ЗАПРОСОВ IRP



Пример обработки IRP-запроса USB-устройством:

1. IRP-запрос формируется функциональным драйвером (Disk.sys) устройства
2. IRP передается Usbstor.sys и одновременно обрабатывается PDO устройства и FDO шины.
3. IRP передается драйверу концентратора (Usbhub.sys) и одновременно обрабатывается PDO устройства и FDO шины.
4. IRP обрабатывается драйверами хост-контроллера USB (Usbuhci.sys)

ЭКВИВАЛЕНТНОЕ ИЗОБРАЖЕНИЕ



СТРУКТУРА ОБЪЕКТА УСТРОЙСТВА

```
typedef struct _DRIVER_OBJECT
{
    CSHORT Type;
    CSHORT Size;
    PDEVICE_OBJECT DeviceObject;
    ULONG Flags;
    PVOID DriverStart;
    ULONG DriverSize;
    PVOID DriverSection;
    PDRIVER_EXTENSION DriverExtension;
    UNICODE_STRING DriverName;
    PUNICODE_STRING HardwareDatabase;
    PFAST_IO_DISPATCH FastIoDispatch;
    PDRIVER_INITIALIZE DriverInit;
    PDRIVER_STARTIO DriverStartIo;
    PDRIVER_UNLOAD DriverUnload;
    PDRIVER_DISPATCH
MajorFunction[IRP_MJ_MAXIMUM_FUNCTION + 1];
} ;
```

ТАБЛИЦА ВЕКТОРОВ ОБЪЕКТА УСТРОЙСТВА

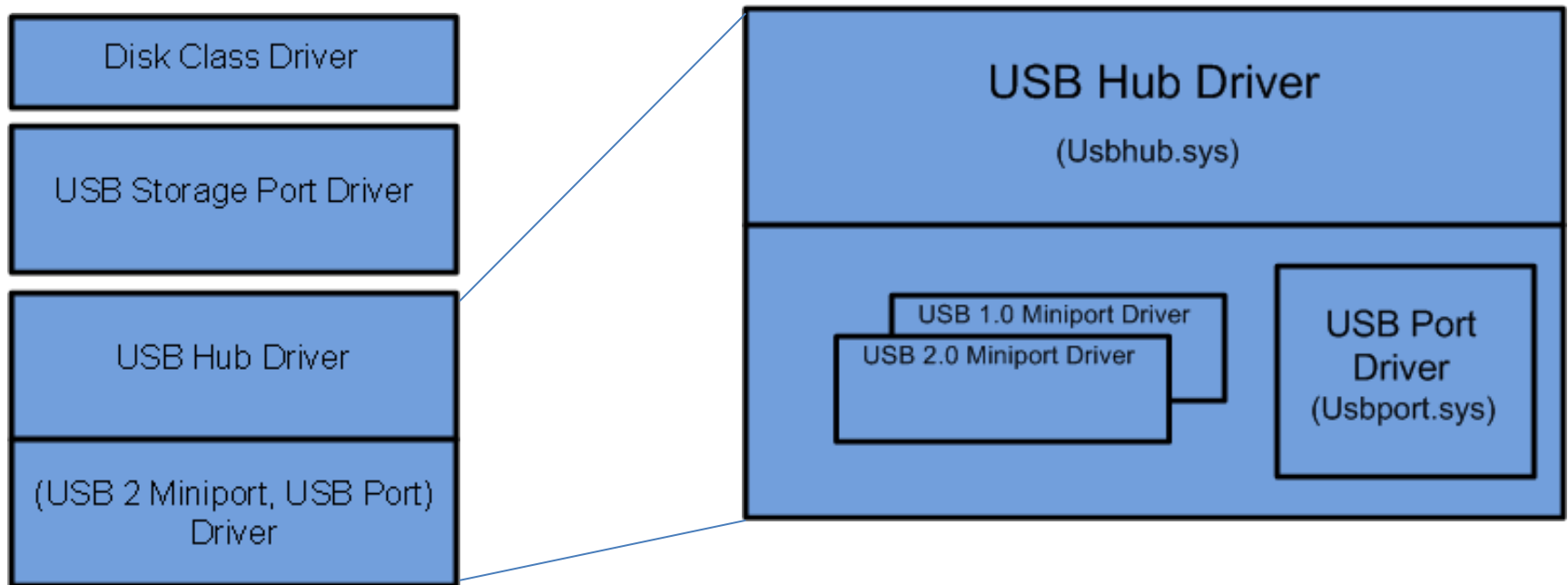
Dispatch routines:

[00] IRP_MJ_CREATE	ffffff880065d49d0	parport!PptDispatchCreateOpen
[01] IRP_MJ_CREATE_NAMED_PIPE	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[02] IRP_MJ_CLOSE	ffffff880065d4a78	parport!PptDispatchClose
[03] IRP_MJ_READ	ffffff880065d4bac	parport!PptDispatchRead
[04] IRP_MJ_WRITE	ffffff880065d4bac	parport!PptDispatchRead
[05] IRP_MJ_QUERY_INFORMATION	ffffff880065d4c40	parport!PptDispatchQueryInformation
[06] IRP_MJ_SET_INFORMATION	ffffff880065d4ce4	parport!PptDispatchSetInformation
[07] IRP_MJ_QUERY_EA	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[08] IRP_MJ_SET_EA	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[09] IRP_MJ_FLUSH_BUFFERS	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[0a] IRP_MJ_QUERY_VOLUME_INFORMATION	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[0b] IRP_MJ_SET_VOLUME_INFORMATION	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[0c] IRP_MJ_DIRECTORY_CONTROL	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[0d] IRP_MJ_FILE_SYSTEM_CONTROL	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[0e] IRP_MJ_DEVICE_CONTROL	ffffff880065d4be8	parport!PptDispatchDeviceControl
[0f] IRP_MJ_INTERNAL_DEVICE_CONTROL	ffffff880065d4c24	parport!PptDispatchInternalDeviceControl
[10] IRP_MJ_SHUTDOWN	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[11] IRP_MJ_LOCK_CONTROL	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[12] IRP_MJ_CLEANUP	ffffff880065d4af4	parport!PptDispatchCleanup
[13] IRP_MJ_CREATE_MAILSLOT	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[14] IRP_MJ_QUERY_SECURITY	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[15] IRP_MJ_SET_SECURITY	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[16] IRP_MJ_POWER	ffffff880065d491c	parport!PptDispatchPower
[17] IRP_MJ_SYSTEM_CONTROL	ffffff880065d4d4c	parport!PptDispatchSystemControl
[18] IRP_MJ_DEVICE_CHANGE	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[19] IRP_MJ_QUERY_QUOTA	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[1a] IRP_MJ_SET_QUOTA	ffffff88001b6ecd4	nt!IopInvalidDeviceRequest
[1b] IRP_MJ_PNP	ffffff880065d4840	parport!PptDispatchPnp

PORT- И MINIPORT-ДРАЙВЕРЫ

Port-драйвер – разновидность драйвера, реализующая абстрактный интерфейс для работы с физическим устройством (напр. Usbport.sys для шины USB)

Miniport-драйвер – драйвер, использующийся как дополнение к port-драйверу и реализующий уточненный интерфейс для работы с устройством (напр. Usbuhci.sys для USB 2.0)



СПАСИБО ЗА ВНИМАНИЕ